Plagiarism Scan Report

2% Plagiarism	2% Exact Match
	0% Partial Match

98% Unique

Words	1000
Characters	7741
Sentences	76
Paragraphs	163
Read Time	5 minute(s)
Speak Time	7 minute(s)

Content Checked For Plagiarism

Employee Payroll and Attendance Management System Page 1
EMPLOYEE PAYROLL AND
ATTENDANCE MANAGEMENT SYSTEM
Industry-Oriented Python Application Development
CSA08 / Programming in Python
Report component Details
Prepared for Assignment submission
Technology Python 3.x, CSV persistence
System type Console-based prototype
Submission date 01 September 2026
Prepared by: _____
Register No.: _____
Class / Section: _____
Employee Payroll and Attendance Management System Page 2
1. Problem Statement
A growing company maintains employee details, daily attendance, leave approvals and payroll in separate spreadsheets and paper registers. This causes duplicated records, slow searches, inconsistent leave balances
and calculation errors in deductions, overtime and bonuses. Monthly reporting is also manual, so management
cannot easily identify attendance leaders, department payroll cost or employees taking excessive leave.
The proposed system is a Python-based Employee Payroll and Attendance Management System that centralizes
these activities. It stores employee data, validates attendance, manages leave and overtime, calculates salary
consistently, produces analysis reports and persists employee records to CSV files.
2. Objective
To develop a reliable prototype that automates attendance and payroll processing while demonstrating core
Python concepts, structured data management, file handling and error control.
• Maintain accurate employee records with unique IDs.
• Record Present, Absent, Leave and Half-day attendance statuses.
• Approve or reject leave requests and retain leave history.
• Calculate deductions, overtime pay and attendance-based bonuses.
• Search employee records and generate management reports.
• Save and reload employee records using CSV files without crashing on invalid input.
3. Requirements and Environment Used

Functional requirements were taken from the assignment brief and mapped to Python modules as follows.
Requirement Implementation
Employee record management Dictionary keyed by Employee ID; dataclass for employee fields.
Attendance and leave Nested dictionary for date/status; list for leave history.
Payroll Salary routine applies absence, excess-leave, half-day, overtime and
bonus rules.
Search and reports Keyword search and department-wise payroll / attendance analysis.
Persistence CSV export and import using the csv module.
Reliability try/except-ready validation and custom exceptions.
Environment: Python 3.x; standard library modules dataclasses, csv, collections, pathlib and datetime; a terminal/command prompt; CSV storage. The design is in-memory for fast access and is appropriate for at least
500 employees in this prototype.
Employee Payroll and Attendance Management System Page 3
4. Design / Proposed Solution
The solution is a layered console application. The service class owns the business rules and data stores. It exposes small, testable methods for employee management, attendance, leave, overtime, salary calculation, searching, reporting and CSV persistence.
4.1 Data Structure Design
Structure Purpose Why it fits
dict[str, Employee] Employee master data O(1) lookup by Employee ID.
dict[str, dict[str, str]] Attendance records Maps an employee to date/status records.
list[dict] Leave history and salary slips Preserves ordered audit-like records.
set[str] Valid attendance statuses Fast membership validation.
tuple / dataclass fields Fixed employee attributes Keeps identity and pay details structured.
4.2 System Architecture
Input layer Business logic Storage / output
Employee, attendance, leave and
overtime entries
Validation; salary calculation;
reports; search
CSV employee file; formatted
payslip values; management report
Security and ethics: salary data should be visible only to authorized users, CSV files should be backed up and write access restricted, and all employees must use the same deduction and overtime rules. Input validation prevents accidental data corruption.
5. Algorithm / Pseudocode / Flowchart
5.1 Salary Calculation Pseudocode
FUNCTION calculate_salary(emp_id, month)
VERIFY emp_id exists
basic_pay ¬ employee.basic_pay
month_records ¬ attendance records for month
absent_days, leave_days, half_days, present_days ¬ count statuses
per_day_pay ¬ basic_pay / 30
deduction ¬ absent_days × per_day_pay
deduction ¬ deduction + max(0, leave_days - allowed_leaves) × per_day_pay
deduction ¬ deduction + half_days × per_day_pay × 0.5
overtime_pay ¬ overtime_hours × overtime_rate
bonus ¬ 5% of basic_pay if attendance rate ³ 95%, else 0
net_salary ¬ basic_pay - deduction + overtime_pay + bonus
SAVE salary slip and RETURN it
5.2 Attendance Validation Pseudocode
FUNCTION mark_attendance(emp_id, date, status)
VERIFY emp_id exists

IF status NOT IN {Present, Absent, Leave, Half-day}
RAISE InvalidAttendanceStatusError
attendance[emp_id][date] ¬ status
RETURN True

5.3 Flowchart

Start
Add / search employee
Mark attendance; process leave and overtime
Validate data; calculate salary
Generate report / CSV output; End

Employee Payroll and Attendance Management System Page 4

6. Implementation / Source Code

The following Python implementation satisfies the specified modules. It generates IDs such as EMP001, uses a custom InvalidAttendanceStatusError, records data in dictionaries/lists, and saves the employee master file to
CSV.

Source File: payroll_attendance_system.py

```
001  """Employee Payroll and Attendance Management System - prototype."""
002  from __future__ import annotations
003
004  import csv
005  from collections import defaultdict
006  from dataclasses import dataclass, asdict
007  from datetime import date
008  from pathlib import Path
009
010
011  class InvalidAttendanceStatusError(ValueError):
012      """Raised when attendance is not a permitted status."""
013
014
015  class InvalidEmployeeError(KeyError):
016      """Raised when an employee id cannot be found."""
017
018
019  VALID_STATUSES = {"Present", "Absent", "Leave", "Half-day"}
020  ALLOWED_LEAVES = 2
021  OVERTIME_RATE = 150.0
022
023
024  @dataclass
025  class Employee:
026      employee_id: str
027      name: str
028      department: str
029      designation: str
030      basic_pay: float
031
032
033  class PayrollAttendanceSystem:
034      def __init__(self) -> None:
035          self.employees: dict[str, Employee] = {}
036          self.attendance: dict[str, dict[str, str]] = defaultdict(dict)
037          self.leave_history: list[dict] = []
```

```
038         self.overtime: dict[str, dict[str, float]] = defaultdict(dict)
039         self.salary_slips: list[dict] = []
040
041     def add_employee(self, name: str, department: str, designation: str, basic_pay: float) -> str:
042         if basic_pay <= 0:</mark>
043             raise ValueError("Basic pay must be positive.")
044         employee_id = f"EMP{len(self.employees) + 1:03d}"
045         self.employees[employee_id] = Employee(employee_id, name.strip().title(),
department.strip().title(), designation.strip().title(), basic_pay)
046         return employee_id
047
048     def _require_employee(self, employee_id: str) -> None:
049         if employee_id not in self.employees:
050             raise InvalidEmployeeError(f"Unknown employee ID: {employee_id}")
051
052     def mark_attendance(self, employee_id: str, day: str, status: str) -> bool:
053         self._require_employee(employee_id)
054         status = status.strip().title()
055         if status not in VALID_STATUSES:
056             raise InvalidAttendanceStatusError(f"Invalid status '{status}'. Use: {sorted(VALID_STATUSES)}")
057         self.attendance[employee_id][day] = status
058         return True
059
060     def request_leave(self, employee_id: str, day: str, reason: str, approved: bool) -> None:
061         self._require_employee(employee_id)
062         record = {"employee_id": employee_id, "date": day, "reason": reason.strip(), "status": "Approved" if
approved else
```

Matched Source

Similarity 1%

Title:Design Model of Payroll System Integrated with Attendance ...

Design and Implementation of Employee Attendance and Payroll Information System Using Desktop-Based Application

Check

Similarity 1%

Title:Source code

... 058 return true; 059 } else { 060 Set<String> thisPart = getParts().get(i); 061 062 for (String token : thisPart) { 063 if (token.equals(WILDCARD_TOKEN)) ...

Check
